Dealing with Outliers

Introduction

The difference between a good and an average machine learning model is often its ability to clean data. One of the biggest challenges in data cleaning is the identification and treatment of outliers. In simple terms, outliers are observations that are significantly different from other data points. Even the best machine learning algorithms will underperform if outliers are not cleaned from the data because outliers can adversely affect the training process of a machine learning algorithm, resulting in a loss of accuracy.

In this guide, you will learn about techniques for outlier identification and treatment in Python.

Data

In this guide, we will be using a fictitious dataset of loan applications containing 600 observations and 6 variables:

1. Income - Annual income of the applicant (in US dollars)
2. Loan_amount - Loan amount (in US dollars) for which the application was submitted
3. Term_months - Tenure of the loan (in months)
4. Credit_score - Whether the applicant's credit score was good ("1") or not ("0")
5. Age - The applicant’s age in years
6. Approval_status - Whether the loan application was approved ("1") or not ("0")

Let's start by loading the required libraries and the data.

```
1# Import required libraries
2import pandas as pd
3import numpy as np
4import matplotlib.pyplot as plt
5
6# Reading the data
7df = pd.read_csv("data_out.csv")
8print(df.shape)
9print(df.info())
```

python

Output:

```
1   (600, 6)
2   <class 'pandas.core.frame.DataFrame'>
```

```
3   RangeIndex: 600 entries, 0 to 599
4   Data columns (total 6 columns):
5   Income            600 non-null int64
6   Loan_amount       600 non-null int64
7   Term_months       600 non-null int64
8   Credit_score      600 non-null int64
9   approval_status   600 non-null int64
10   Age              600 non-null int64
11   dtypes: int64(6)
12   memory usage: 28.2 KB
13   None
```

The above output shows that there are 600 observations of 6 variables. All the variables have 600 records, indicating that there is no missing value in the data.

Outlier Identification

There can be many reasons for the presence of outliers in the data. Sometimes the outliers may be genuine, while in other cases, they could exist because of data entry errors. It is important to understand the reasons for the outliers before cleaning them.
We will start the process of finding outliers by running the summary statistics on the variables. This is done using the *describe()* function below, which provides a statistical summary of all the quantitative variables.

```
1df.describe()
```

python
Output:

```
1|       | Income          | Loan_amount   | Term_months   | Credit_score    | approval_status  | Age             |
2|------- |---------------  |-------------  |-------------  |-------------    |----------------  |-----------  |
3| count          | 600.000000     | 600.000000     | 600.00000      | 600.000000     | 600.000000      | 600.000000     |
4| mean           | 7210.720000    | 161.571667     | 367.10000      | 0.788333       | 0.686667        | 51.766667      |
5| std    | 8224.445086    | 93.467598      | 63.40892       | 0.408831       | 0.464236        | 21.240704      |
```

```
6| min    | 200.000000      | 10.000000       | 36.00000        | 0.000000        |
0.000000          | 0.000000        |
7| 25%            | 3832.500000     | 111.000000      | 384.00000       | 1.000000        |
0.000000          | 36.000000       |
8| 50%            | 5075.000000     | 140.000000      | 384.00000       | 1.000000        |
1.000000          | 51.000000       |
9| 75%            | 7641.500000     | 180.500000      | 384.00000       | 1.000000        |
1.000000          | 64.000000       |
10| max           | 108000.000000           | 778.000000      | 504.00000       |
1.000000          | 1.000000        | 200.000000      |
```

Looking at the 'Age' variable, it is easy to detect outliers resulting from incorrect data. The minimum and maximum ages are 0, and 200, respectively. These are incorrect, and we will treat them later in the guide. These outliers were easy to detect, butthat will not always be the case. In other cases, mathematical and visualization techniques must be used. These techniques are discussed in the following sections.

Identifying Outliers with Interquartile Range (IQR)

The interquartile range (IQR) is a measure of statistical dispersion and is calculated as the difference between the 75th and 25th percentiles. It is represented by the formula $IQR = Q3 - Q1$. The lines of code below calculate and print the interquartile range for each of the variables in the dataset.

```
1Q1 = df.quantile(0.25)
2Q3 = df.quantile(0.75)
3IQR = Q3 - Q1
4print(IQR)
```

python
Output:

```
1   Income           3809.0
2   Loan_amount        69.5
3   Term_months         0.0
4   Credit_score        0.0
5   approval_status     1.0
6   Age                28.0
7   dtype: float64
```

The above output prints the IQR scores, which can be used to detect outliers. The code below generates an output with the 'True' and 'False' values. Points where the values are 'True' represent the presence of the outlier.

```
print(df < (Q1 - 1.5 * IQR)) |(df > (Q3 + 1.5 * IQR))
```

python
Output:

	Income	Loan_amount	Term_months	Credit_score	approval_status	Age
0	False	False	False	False	False	False
1	False	False	True	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
5	False	False	False	False	False	False
6	False	False	False	False	False	False
7	False	False	False	False	False	False
8	False	False	False	False	False	False
9	False	False	False	False	False	False
10	False	False	False	False	False	False
11	False	False	False	False	False	False
12	False	False	True	True	False	False
13	False	False	False	False	False	False
14	False	False	False	False	False	False
15	False	False	False	True	False	False
16	False	False	False	False	False	False
17	False	False	False	False	False	False
18	False	False	False	False	False	False
19	False	False	False	True	False	False
20	False	False	False	False	False	False
21	False	False	False	False	False	False
22	False	False	False	True	False	False
23	False	False	False	False	False	False

26	24	False	False	False	False	False	False
27	25	False	False	False	False	False	False
28	26	False	False	False	False	False	False
29	27	False	False	True	True	False	False
30	28	False	False	False	False	False	False
31	29	False	False	False	True	False	False
32	..	...	...	...	...	...	...
33	570	False	False	False	False	False	False
34	571	False	False	False	False	False	False
35	572	False	False	False	False	False	False
36	573	False	False	False	True	False	False
37	574	False	False	False	False	False	False
38	575	False	False	False	True	False	False
39	576	False	False	False	False	False	False
40	577	False	False	False	True	False	False
41	578	False	False	False	False	False	False
42	579	False	False	False	False	False	False
43	580	False	False	False	False	False	False
44	581	False	False	False	True	False	False
45	582	False	False	False	True	False	False
46	583	False	False	False	False	False	False
47	584	False	False	True	False	False	False
48	585	False	False	False	False	False	False
49	586	False	False	False	False	False	False
50	587	False	False	False	False	False	False
51	588	False	False	False	False	False	False
52	589	False	False	False	True	False	False
53	590	False	False	False	False	False	False
54	591	False	False	False	False	False	False
55	592	False	False	False	True	False	False

```
56  593  False  False  False  False  False  False
57  594  False  False  False  False  False  False
58  595  False  False  False  False  False  False
59  596  False  False  False  False  False  False
60  597  False  False  False  False  False  False
61  598  False  False  False   True  False  False
62  599  False  False  False  False  False  False
63
64  [600 rows x 6 columns]
```

Identifying Outliers with Skewness

Several machine learning algorithms make the assumption that the data follow a normal (or Gaussian) distribution. This is easy to check with the *skewness value*, which explains the extent to which the data is normally distributed. Ideally, the skewness value should be between -1 and +1, and any major deviation from this range indicates the presence of extreme values.
The first line of code below prints the skewness value for the 'Income' variable, while the second line prints the summary statistics.

```
1print(df['Income'].skew())
2df['Income'].describe()
```

python
Output:

```
1   6.499
2
3   count      600.000000
4   mean      7210.720000
5   std      8224.445086
6   min       200.000000
7   25%       3832.500000
8   50%       5075.000000
9   75%       7641.500000
10   max     108000.000000
11   Name: Income, dtype: float64
```

The skewness value of 6.5 shows that the variable 'Income' has a right-skewed distribution, indicating the presence of extreme higher values. The maximum 'Income' value of USD 108,000 proves this point.

Identifying Outliers with Visualization

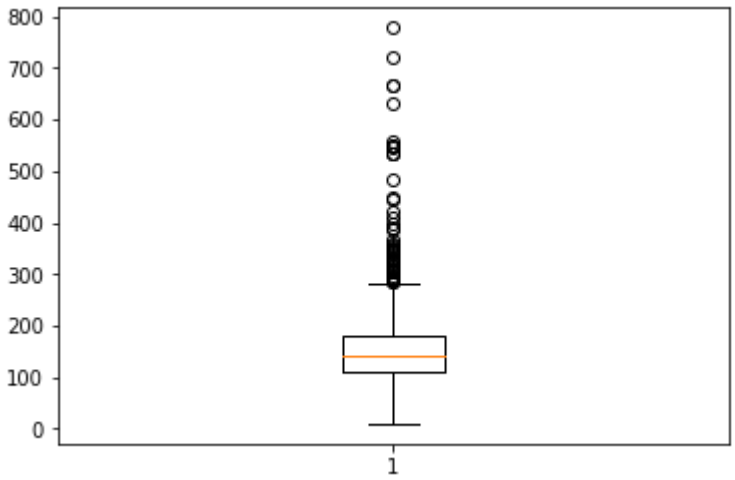
In the previous section, we used quantitative methods for outlier identification. This can also be achieved with visualization. Some of the common plots used for outlier detection are discussed below.

1. Box Plot

The box plot is a standardized way of displaying the distribution of data based on the five-number summary (minimum, first quartile (Q1), median, third quartile (Q3), and maximum). It is often used to identify data distribution and detect outliers. The line of code below plots the box plot of the numeric variable 'Loan_amount'.

```
1plt.boxplot(df["Loan_amount"])
2plt.show()
```

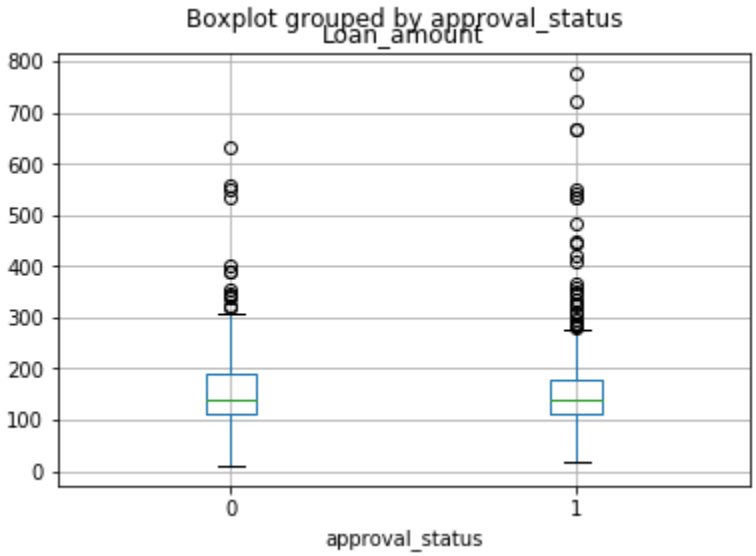
python
Output:



In the above output, the circles indicate the outliers, and there are many. It is also possible to identify outliers using more than one variable. We can modify the above code to visualize outliers in the 'Loan_amount' variable by the approval status.

```
1df.boxplot(column='Loan_amount', by='approval_status')
```

python
Output:



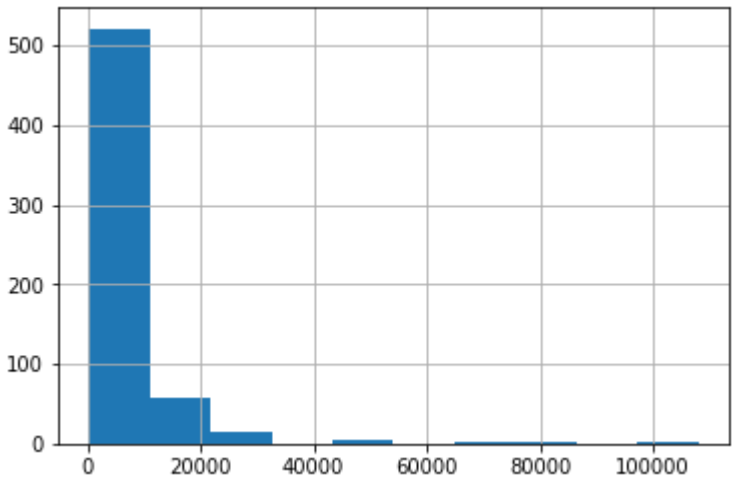
The output shows that the number of outliers is higher for approved loan applicants (denoted by the label '1') than for rejected applicants (denoted by the label '0').

2. Histogram

A histogram is used to visualize the distribution of a numerical variable. An outlier will appear outside the overall pattern of distribution. The line of code below plots a histogram of the 'Income' variable, using the *hist()* function.

```
1df.Income.hist()
```

python
Output:



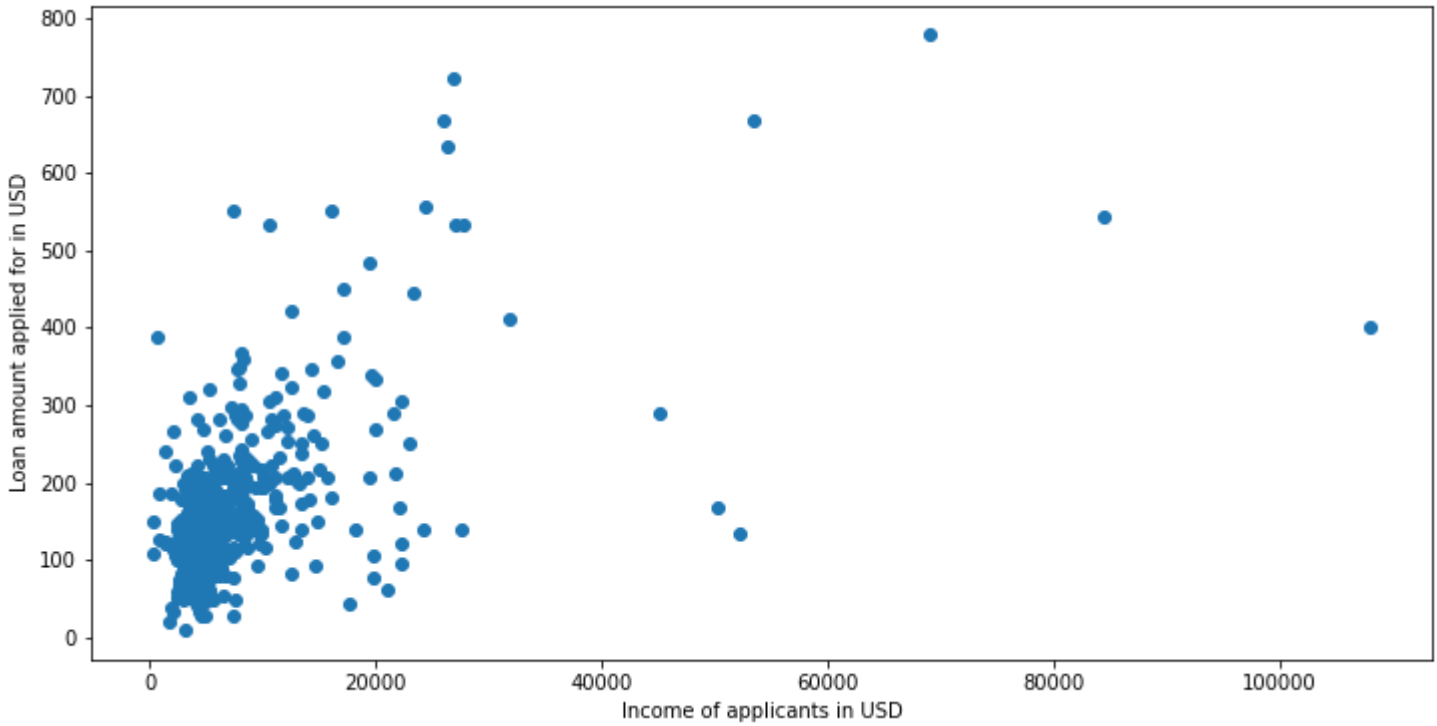
The above chart shows that the distribution is right-skewed, and there are extreme higher values at the right of the histogram. This step can be repeated for other variables as well.

3. Scatterplot

A scatterplot visualizes the relationship between two quantitative variables. The data are displayed as a collection of points, and any points that fall outside the general clustering of the two variables may indicate outliers. The lines of code below generate a scatterplot between the variables 'Income' and 'Loan_amount'.

```
fig, ax = plt.subplots(figsize=(12,6))
ax.scatter(df['Income'], df['Loan_amount'])
ax.set_xlabel('Income of applicants in USD')
ax.set_ylabel('Loan amount applied for in USD')
plt.show()
```

python
Output:



The above chart indicates that most of the data points are clustered in the lower half of the plot. The points located to the extreme right of the x-axis or the y-axis indicate outliers.

Outlier Treatment

In the previous sections, we learned about techniques for outlier detection. However, this is only half of the task. Once we have identified the outliers, we need to treat them. There are several techniques for this, and we will discuss the most widely used ones below.

- **Do nothing**

Sometimes missing data is very less number of rows (say less than 3%) then we can simply ignore the missing data. There is no hard rule to keep the missing data it depends on us

- **Quantile-based Flooring and Capping**

In this technique, we will do the flooring (e.g., the 10th percentile) for the lower values and capping (e.g., the 90th percentile) for the higher values. The lines of code below print the 10th and 90th percentiles of the variable 'Income', respectively. These values will be used for quantile-based flooring and capping.

```
1print(df['Income'].quantile(0.10))
2print(df['Income'].quantile(0.90))
```

python
Output:

```
1 2960.1
2 12681.0
```

Now we will remove the outliers, as shown in the lines of code below. Finally, we calculate the skewness value again, which comes out much better now.

```
1df["Income"] = np.where(df["Income"] <2960.0, 2960.0,df['Income'])
2df["Income"] = np.where(df["Income"] >12681.0, 12681.0,df['Income'])
3print(df['Income'].skew())
```

python
Output:

```
1  1.04
```

- **Trimming**

In this method, we completely remove data points that are outliers. Consider the 'Age' variable, which had a minimum value of 0 and a maximum value of 200. The first line of code below creates an index for all the data points where the age takes these two values. The second line drops these index rows from the data, while the third line of code prints summary statistics for the variable.

After trimming, the number of observations is reduced from 600 to 594, and the minimum and maximum values are much more acceptable.

```
index = df[(df['Age'] >= 100)|(df['Age'] <= 18)].index
df.drop(index, inplace=True)
df['Age'].describe()
```

python
Output:

```
count   594.000000
mean     50.606061
std      16.266324
min      22.000000
25%      36.000000
50%      50.500000
75%      64.000000
max      80.000000
Name: Age, dtype: float64
```

- **IQR Score**

This technique uses the IQR scores calculated earlier to remove outliers. The rule of thumb is that anything not in the range of *(Q1 - 1.5* IQR) and (Q3 + 1.5 *IQR)* is an outlier, and can be removed. The first line of code below removes outliers based on the IQR range and stores the result in the data frame 'df_out'. The second line prints the shape of this data, which comes out to be 375 observations of 6 variables. This shows that for our data, a lot of records get deleted if we use the IQR method.

```
df_out = df[~((df < (Q1 - 1.5 * IQR)) |(df > (Q3 + 1.5 * IQR))).any(axis=1)]
print(df_out.shape)
```

python
Output:

```
(375, 6)
```

- **Log Transformation**

Transformation of the skewed variables may also help correct the distribution of the variables. These could be logarithmic, square root, or square transformations. The most common is the logarithmic transformation, which is done on the 'Loan_amount' variable in the first line of code

below. The second and third lines of code print the skewness value before and after the transformation.

```
df["Log_Loanamt"] = df["Loan_amount"].map(lambda i: np.log(i) if i > 0 else 0)
print(df['Loan_amount'].skew())
print(df['Log_Loanamt'].skew())
```

python
Output:

```
2.8146019248106815
-0.17792641310111373
```

The above output shows that the skewness value came down from 2.8 to -0.18, confirming that the distribution has been treated for extreme values.

- **Replacing Outliers with Median Values**

In this technique, we replace the extreme values with median values. It is advised to not use mean values as they are affected by outliers. The first line of code below prints the 50th percentile value, or the median, which comes out to be 140. The second line prints the 95th percentile value, which comes out to be around 326. The third line of code below replaces all those values in the 'Loan_amount' variable, which are greater than the 95th percentile, with the median value. Finally, the fourth line prints summary statistics after all these techniques have been employed for outlier treatment.

```
print(df['Loan_amount'].quantile(0.50))
print(df['Loan_amount'].quantile(0.95))
df['Loan_amount'] = np.where(df['Loan_amount'] > 325, 140, df['Loan_amount'])
df.describe()
```

python
Output:

```
140.0
325.7500000000001


| | Income | Loan_amount | Term_months | Credit_score | approval_status | Age | Log_Loanamt |
|---|---|---|---|---|---|---|---|
```

7	count	594.000000	594.000000	594.000000	594.000000	594.000000	594.000000	594.000000
8	mean	6112.375421	144.289562	366.929293	0.787879	0.688552	50.606061	4.957050
9	std	3044.257269	53.033735	63.705994	0.409155	0.463476	16.266324	0.494153
10	min	2960.000000	10.000000	36.000000	0.000000	0.000000	22.000000	2.302585
11	25%	3831.500000	111.000000	384.000000	1.000000	0.000000	36.000000	4.709530
12	50%	5050.000000	140.000000	384.000000	1.000000	1.000000	50.500000	4.941642
13	75%	7629.000000	171.000000	384.000000	1.000000	1.000000	64.000000	5.192957
14	max	12681.000000	324.000000	504.000000	1.000000	1.000000	80.000000	6.656727